PORTALDOT HACKER HOUSE 2026 · MODULAR CLOUD & MIDDLEWARE

Portal Meter

Pay-per-request API settlement on Portaldot V3,
built on both of its contract VMs.

github.com/swiftbotlab/portal-meter · swiftbotlab.github.io/portal-meter/demo

The problem

- APIs are now bought by **programs**: apps, bots, AI agents, indexers. They want to pay **per call**, cents or less.
- API keys + monthly plans don't fit a client that needs 40 calls today and none tomorrow.
- "Pay first" protocols (HTTP 402 / x402) move money but give the buyer **no recourse** when the call fails.
- And no way to know, before paying, whether a provider actually delivers.

What a machine buyer needs
price up front · pay only for what is served · automatic refund when it isn't · a public track record of the seller

Portal Meter

1. **GET** → `402` with a quote (price, token, endpoint, nonce)
2. Client pays **one request** into escrow: `MeterSettlement.pay()` on Portaldot's EVM; the request id is bound to the payer on-chain
3. Client retries with a signed proof → served **once**, with a provider-signed receipt
4. Provider **settles** (funds to payee) or **refunds**; the buyer can reclaim after a delay if the provider does nothing
5. The outcome is **anchored in an ink! registry** in the same transaction; buyers confirm or dispute
6. Anyone verifies a receipt: **ACCEPT / REJECT** with every check (CLI and browser)

Why Portaldot is essential

Native assets are ERC-20s

Every `pallet-assets` token is reachable from Solidity at a precompile address. Endpoints are priced in a native asset; no wrapped token to deploy or trust.

Two VMs, one identity

Escrow in Solidity (pallet-revive), provider registry in ink! (pallet-contracts). Both sides key accounts by the same EVM address, using the runtime's own mapping rule.

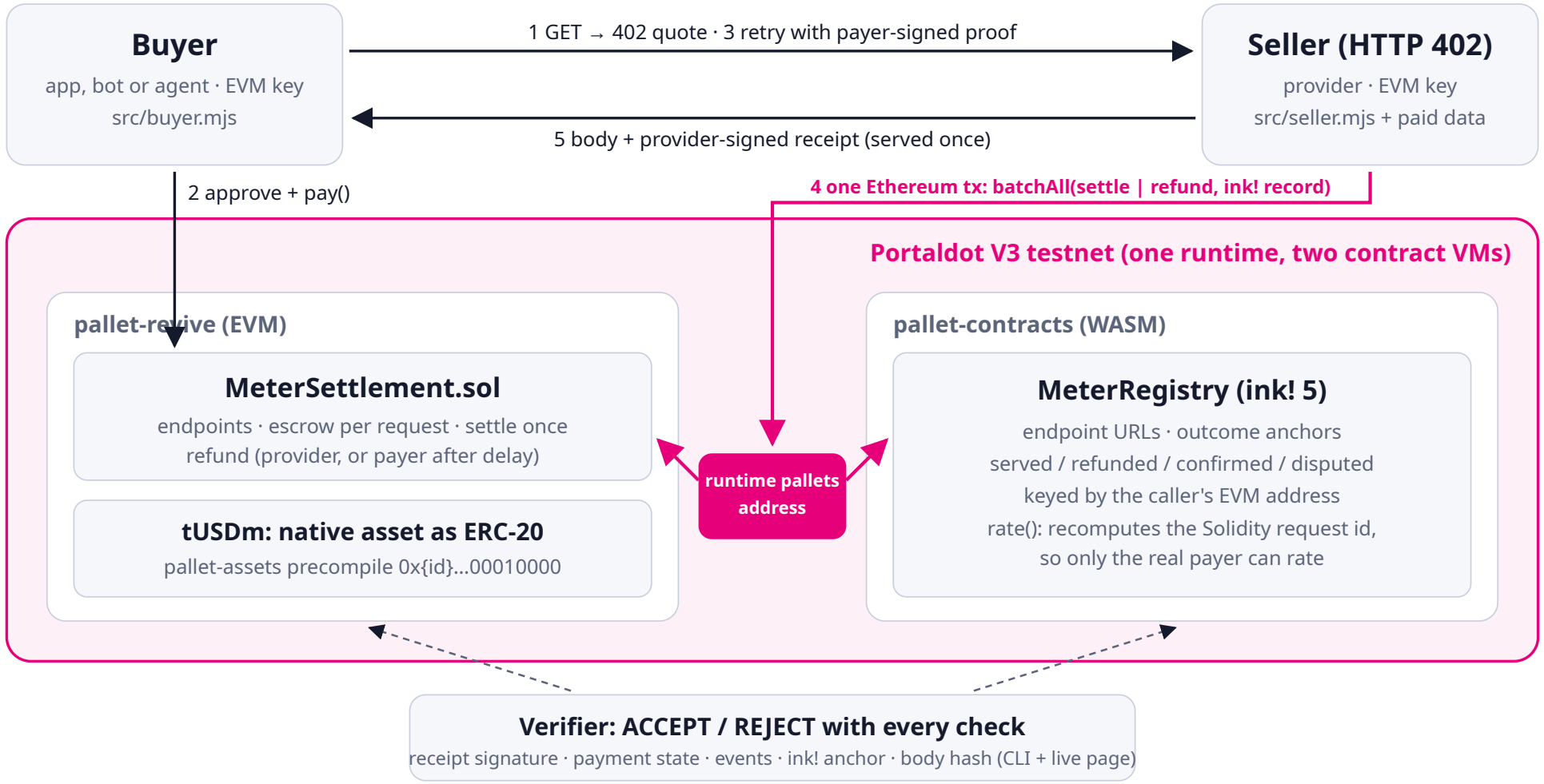
One transaction across both VMs

An Ethereum transaction to the runtime pallets address carries `utility.batchAll([revive.call(settle), contracts.call(record)])`: settlement and anchor succeed or fail together. A single-VM chain cannot do this.

Ink! checks EVM payers natively

The registry recomputes the Solidity request id (keccak + ABI) from the caller's address, so only the real payer can rate a request, with no bridge or oracle.

Architecture



Tested, and checked on the live chain

20

Solidity tests

unit · fuzz · invariant

8

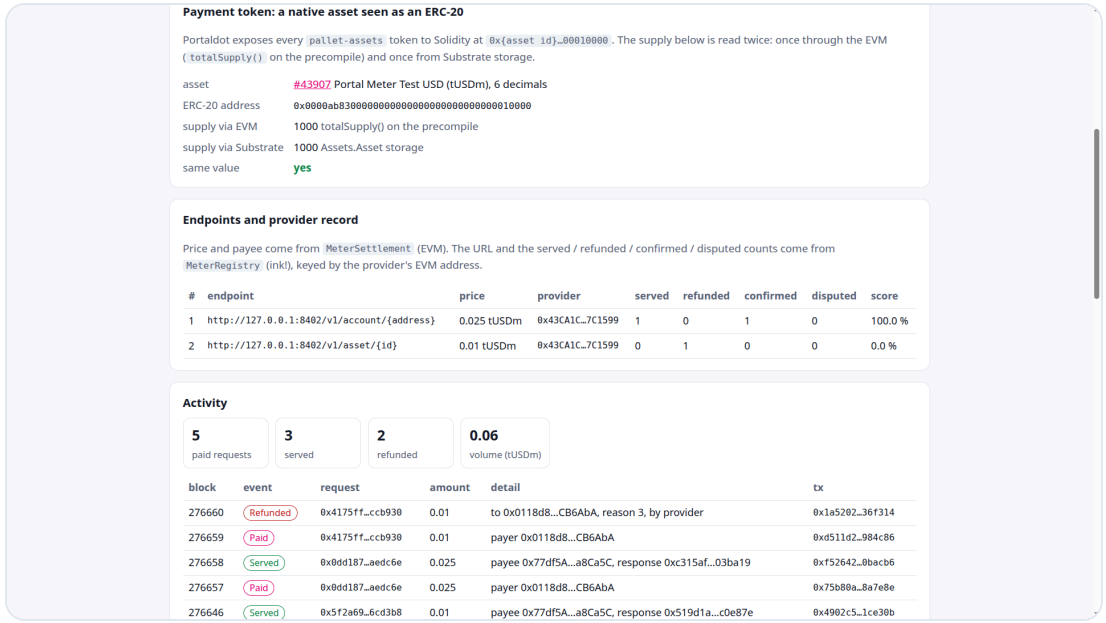
ink! unit tests

25

Node tests incl.

end to end on a chain

- CI on every push: Foundry, ink! release build (cargo-contract 5.0.3), Node end to end
- 14 negative checks on the live contracts: replays, strangers and tampered receipts are refused
- Live page reads both VMs from the browser and verifies receipts
- Open source, MIT: github.com/swiftbotlab/portal-meter



Payment token: a native asset seen as an ERC-20

Portaldot exposes every `pallet-assets` token to Solidity at `0x{asset id}…00010000`. The supply below is read twice: once through the EVM (`totalSupply()` on the precompile) and once from Substrate storage.

asset	#43907 Portal Meter Test USD (tUSDm), 6 decimals
ERC-20 address	0x0000ab830000000000000000000000000010000
supply via EVM	1000 totalSupply() on the precompile
supply via Substrate	1000 Assets.Asset storage
same value	yes

Endpoints and provider record

Price and payee come from `MeterSettlement` (EVM). The URL and the served / refunded / confirmed / disputed counts come from `MeterRegistry` (ink!), keyed by the provider's EVM address.

#	endpoint	price	provider	served	refunded	confirmed	disputed	score
1	http://127.0.0.1:8402/v1/account/{address}	0.025 tUSDm	0x43CA1C…7C1599	1	0	1	0	100.0 %
2	http://127.0.0.1:8402/v1/asset/{id}	0.01 tUSDm	0x43CA1C…7C1599	0	1	0	0	0.0 %

Activity

5	3	2	0.06
paid requests	served	refunded	volume (tUSDm)

block	event	request	amount	detail	tx
276660	Refunded	0x4175ff…ccb930	0.01	to 0x0118d8…CB6AbA, reason 3, by provider	0x1a5202…36f314
276659	Paid	0x4175ff…ccb930	0.01	payer 0x0118d8…CB6AbA	0xd511d2…984c86
276658	Served	0x0dd187…aedc6e	0.025	payee 0x77df5A…a8Ca5C, response 0xc315af…03ba19	0xf52642…0bacb6
276657	Paid	0x0dd187…aedc6e	0.025	payer 0x0118d8…CB6AbA	0x75b80a…8a7e8e
276646	Served	0x5f2a69…6cd3b8	0.01	payee 0x77df5A…a8Ca5C, response 0x519d1a…c0e87e	0x4902c5…1ce30b

Live on the PortaldotV3 testnet

MeterSettlement (Solidity, EVM)	`0x9E7Ad2Aa75…ef97e5781e`	deploy 0x1adf1c42…15731227 · block 276611
MeterRegistry (ink!, WASM)	`5DHgN8JZRpyo…K1K2Vd8Ku4`	instantiate 0xef17ab33…97bd60e0 · block 276655
Native asset #43907 tUSDm	`0x0000ab8300…0000010000`	ERC-20 precompile address
Endpoints listed in ink! from the provider's Ethereum key	`0x7c14d9debd…12f6cc569b`	block 276656
Paid request: buyer pays into escrow	`0x75b80a21c8…dae28a7e8e`	block 276657
Served: settle + ink! anchor, one Ethereum tx	`0xf5264278d3…1e7e0bacb6`	block 276658
Refunded: refund + ink! anchor, one Ethereum tx	`0x1a52029d1a…d68a36f314`	block 276660
Buyer confirms in ink! from its Ethereum key	`0x91d0a18a7a…7970cd3297`	block 276665

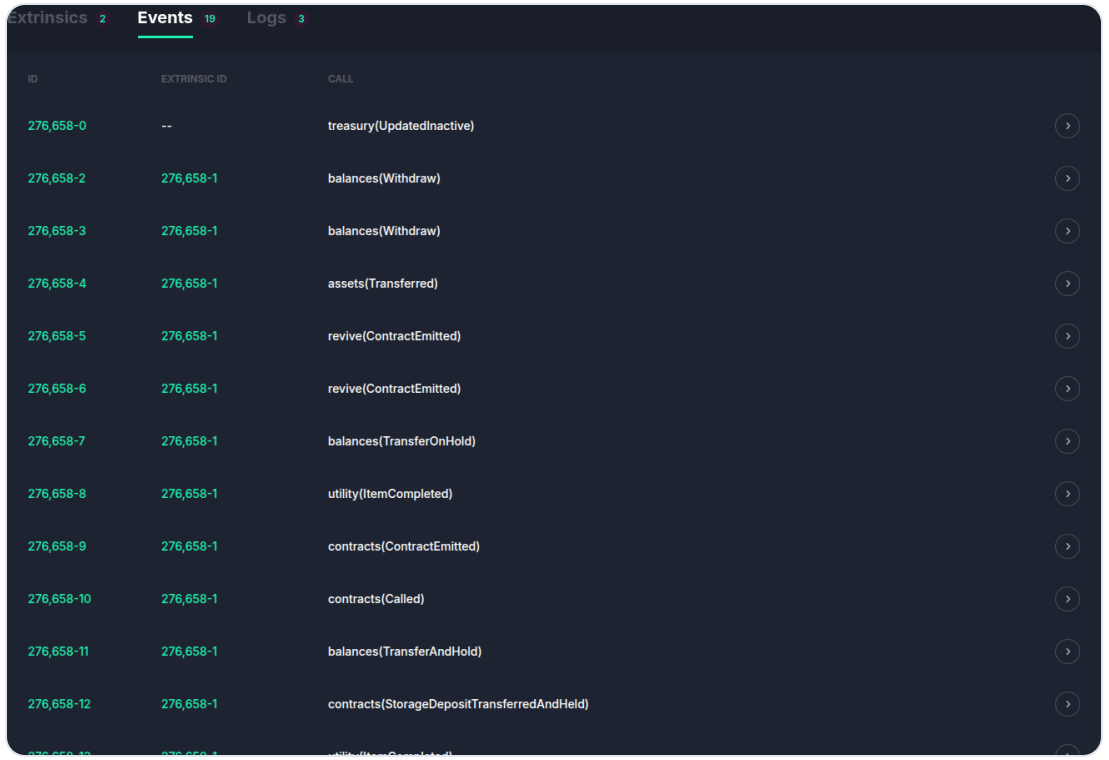
All transactions with blocks: deployments/testnet.json · live page: swiftbotlab.github.io/portal-meter/demo · explorer https://scanner-testnet-v3.portaldot.dev

One transaction, both VMs

The provider's Ethereum key signs **one** transaction to the runtime pallets address:

`utility.batchAll([revive.call(settle), contracts.call(record)])`

- The buyer's escrow moves to the payee (`assets.Transferred`, `revive.ContractEmitted`)
- The ink! registry anchors the outcome (`contracts.ContractEmitted`, `contracts.Called`)
- All or nothing: `utility.BatchCompleted`
- Block 276,658 on the testnet explorer, events tab



Who pays, and how it grows

- **Users:** data/API providers on Portaldot (indexers, RPC and analytics services, AI tool endpoints) and the apps and agents that call them.
- **First product:** the paid Portaldot data API in this repo (cross-VM account and asset views): a real endpoint other builders can call per request.
- **Revenue (planned):** a small protocol fee on settlement (basis points, per deployment; not in the testnet contract yet) and a hosted seller for providers who do not want to run one.
- **Growth:** drop-in middleware for Express/Fastify, listing in the Portaldot ecosystem directory, wallet support for 402 quotes.

Roadmap, and why keep supporting it

Next 4 weeks

npm middleware package · batch settlement (many requests per transaction) · provider bond held in the ink! registry, slashable on disputes · mainnet migration review

Next quarter

prepaid credit channels for high-frequency callers · explorer / wallet integration of receipts · more paid Portaldot data endpoints

What Portaldot gets

open-source middleware every API builder can reuse · real fee-paying transactions on both VMs · a reference for cross-VM design (documented, tested, measured)

Team

Swift Bot Labs, a small studio building payment and API infrastructure. Builds on our MIT-licensed HSK Meter (Oct 2026); this repository was written for Portaldot during the Hacker House.